Virtual Threads and Structured Concurrency

8 min read Updated 28 Jul 2026 Topic 51 of 65

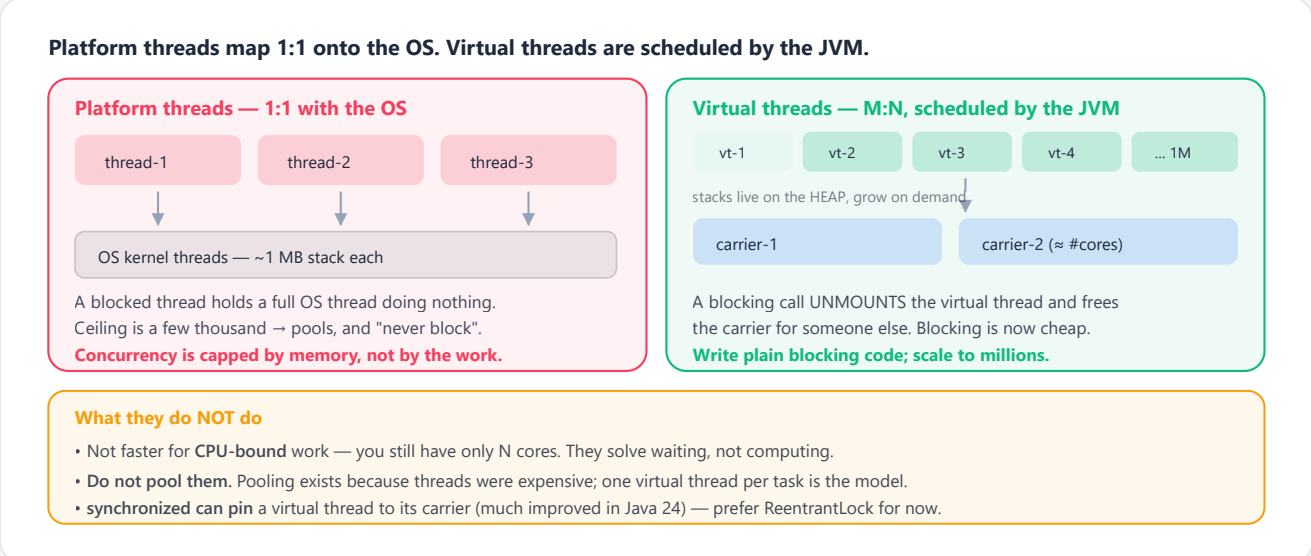
Bookmark Mark done

Why it matters

Virtual threads (Java 21) are the most consequential change to Java concurrency since Java 5. They invalidate the assumption behind thread pools, reactive programming and much of Chapter 8's advice — not by making that advice wrong, but by removing the constraint it was working around. "How would you write this on Java 21?" is now a standard senior follow-up, and the answer is usually "the simple blocking way, which finally scales".

The concept, simply

A platform thread maps one-to-one onto an OS thread: roughly a megabyte of stack, kernel scheduling, a ceiling of a few thousand. A **virtual thread** is scheduled by the JVM onto a small pool of *carrier* platform threads, with its stack on the heap and grown on demand.



The API is deliberately unchanged — they are still `Thread`, still `Runnable`, still work with `try`/`finally`, thread dumps and debuggers. That is the point: existing blocking code scales without being rewritten.

Code examples

JAVA Copy

```java
// Creating them
Thread.ofVirtual().start(() -> handle(request));
Thread.startVirtualThread(() -> handle(request));

// The executor you will actually use — a NEW virtual thread per task.
// Do not pool them; pooling exists because threads were expensive.
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var request : requests) {
        executor.submit(() -> handle(request));    // blocking code is fin
    }
}   // close() waits for all tasks — ExecutorService is AutoCloseable in 1

// A million concurrent tasks, each blocking for a second:
try (var e = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 1_000_000).forEach(i ->
        e.submit(() -> { Thread.sleep(1000); return i; }));
}
// Completes in about a second. With platform threads this is impossible.
```

What changes in ordinary code — and what does not:

JAVA Copy

```java
// ✓ Plain blocking code is now the RIGHT answer for I/O fan-out:
try (var scope = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<Profile> p = scope.submit(() -> fetchProfile(id));   // blockin
    Future<List<Order>> o = scope.submit(() -> fetchOrders(id)); // blocki
    return new Dashboard(p.get(), o.get());
```

```
// far less machinery, and stack traces that actually mean something.

// ⚠ ThreadLocal still works but is now a memory concern: a million virtu
//   threads means a million copies. Scoped values (preview) address this.

// ⚠ Do NOT pool virtual threads, and do not size anything:
// ✗ Executors.newFixedThreadPool(200, Thread.ofVirtual().factory());
```

Structured concurrency (preview in 21, still evolving) — the companion feature:

JAVA Copy

```
// The problem it solves: with an executor, if one subtask fails the other
// keep running, and there is no relationship between the tasks and the
// method that started them.
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Supplier<Profile> profile = scope.fork(() -> fetchProfile(id));
    Supplier<List<Order>> orders = scope.fork(() -> fetchOrders(id));

    scope.join();                // wait for all
    scope.throwIfFailed();       // propagate the first failure

    return new Dashboard(profile.get(), orders.get());
}   // ✓ If EITHER fails, the other is cancelled automatically. If the cal
    //   is interrupted, both are cancelled. The task tree matches the cod
```

Under the hood

Mounting and unmounting is the mechanism. A virtual thread runs on a carrier; when it hits a blocking call the JVM copies its stack to the heap, unmounts it, and gives the carrier to another virtual thread. When the operation completes it is remounted, possibly on a different carrier. The JDK's I/O, locks, and `Thread.sleep` were all reworked to cooperate with this.

Pinning is the caveat to know. In Java 21 a virtual thread inside a `synchronized` block cannot unmount — it *pins* its carrier, so a blocking call there ties up a platform thread

is version-specific — saying so precisely is a good signal.

They do not help CPU-bound work. You still have N cores; virtual threads solve *waiting*, not *computing*. A CPU-bound workload still wants a pool sized to the cores. Knowing which of the two situations you are in is the whole skill.

What they make obsolete: much of the reason for reactive frameworks. Project Reactor and RxJava exist largely to avoid blocking a scarce thread; when blocking is cheap, the reactive style's cost — inverted control flow, unreadable stack traces, difficult debugging — stops being worth paying for most applications. Reactive still wins where you genuinely need backpressure and stream composition. That nuance is a much better answer than "virtual threads killed reactive".

Common mistakes

- **Pooling virtual threads.** Pools exist because threads were expensive. One per task is the model.
- **Expecting CPU-bound speedups.** They solve waiting, not computing.
- **Blocking inside `synchronized` on Java 21.** Pins the carrier — use `ReentrantLock`.
- **Heavy `ThreadLocal` use.** A million threads means a million copies.
- **Assuming the whole stack cooperates.** Some native and JNI calls still pin.
- **Removing all bounds.** Cheap threads do not make a downstream database able to take a million connections — you still need a limit, now usually a `Semaphore`.

Interview lens

> **Q: "What are virtual threads?"**
> A: Lightweight threads scheduled by the JVM rather than the OS. Their stacks live on the heap and grow on demand, so millions are practical, and when one blocks it unmounts from its carrier platform thread, freeing it for other work. The API is unchanged — they are still `Thread` — so ordinary blocking code scales without being rewritten.

A: Blocking stops being expensive, so the "never block a thread" discipline behind thread pools and reactive programming becomes unnecessary for I/O-bound work. I would use `newVirtualThreadPerTaskExecutor`, write straightforward blocking calls, and get readable stack traces and debuggable code. Pools remain right for CPU-bound work, where the constraint is cores rather than thread cost.

Q: "Should you pool virtual threads?"

A: No — pooling exists precisely because platform threads are expensive to create. A virtual thread is cheap, so the model is one per task. If you need to limit concurrency against a downstream resource, use a `Semaphore` to bound the work rather than a pool to bound the threads; those are different concerns that pooling used to conflate.

Q: "What is pinning?"

A: A virtual thread that cannot unmount from its carrier. In Java 21 that happens inside a `synchronized` block, so blocking there ties up a platform thread and can starve the scheduler — the guidance was to prefer `ReentrantLock` in blocking code. It was substantially fixed in Java 24, so the answer depends on the version you are targeting. Some native and JNI frames still pin.

Q: "Do virtual threads make reactive programming obsolete?"

A: For a large share of its use cases, yes — most reactive code exists to avoid blocking a scarce thread, and it pays for that with inverted control flow, poor stack traces and hard debugging. When blocking is cheap that trade stops making sense. Reactive still wins where you genuinely need backpressure, stream composition or event-driven pipelines; it just stops being the default for "call three services concurrently".

A: Tying the lifetime of concurrent subtasks to a syntactic scope, so the task tree matches the code structure. With `StructuredTaskScope`, forked tasks are cancelled automatically if one fails or the caller is interrupted, and the scope cannot be exited until all are complete. It removes the class of bug where a failed request leaves orphaned subtasks running. It is still a preview feature and the API has been evolving.

Cheat-card

- Virtual thread = **JVM-scheduled, heap stack, M:N onto carriers**. Same `Thread` API.
- Blocking **unmounts** the virtual thread and frees the carrier → blocking is cheap.
- `Executors.newVirtualThreadPerTaskExecutor()`. **Never pool them.**
- **No help for CPU-bound work** — still N cores.
- **Pinning**: `synchronized` blocks unmounting in Java 21 → prefer `ReentrantLock`. Fixed in 24.
- `ThreadLocal` × millions = memory; scoped values are the answer.
- Still bound concurrency against downstreams — use a `Semaphore`, not a pool.
- **Structured concurrency**: `StructuredTaskScope` ties subtask lifetime to a scope; auto-cancellation.
- Reduces the case for reactive; does not eliminate it (backpressure, stream composition).

QUICK CHECK — no marks, just for you.

1. Should you pool virtual threads?

- ◯ Yes, otherwise they are never garbage collected
- ◯ Yes, with a fixed pool sized to the cores
- ◯ Yes, but only for I/O work

No — pooling exists because platform threads are expensive to create; a virtual thread is cheap, so one per task is the model

Check answer

2. Do virtual threads make CPU-bound work faster?

- ◯ No — they solve WAITING, not computing; you still have only N cores, so CPU-bound work still wants a pool sized to the cores
- ◯ Yes, because they have smaller stacks
- ◯ Only when combined with parallel streams
- ◯ Yes, because the JVM schedules them more efficiently than the OS

Check answer

3. What is 'pinning' in the context of virtual threads?

- ◯ Preventing a virtual thread from being garbage collected
- ◯ A virtual thread that cannot unmount from its carrier — in Java 21 this happens inside a synchronized block, so blocking there ties up a platform thread
- ◯ Reserving heap space for a virtual thread's stack
- ◯ Assigning a virtual thread to a specific CPU core

Check answer

LEARN MORE

ARTICLES & DOCS

- Virtual threads (JEP 444)
  openjdk.org
- Structured concurrency (JEP 453)
  openjdk.org
- Thread — API docs
  docs.oracle.com

VIDEOS



- Project Loom: Modern Scalable Concurrency for the Java Platform — Ron Pressler
  CHariotSolutions



- Ron Pressler - Loom: Bringing Lightweight Threads and Delimited Continuations to the JVM
  Curry On!

Hand-picked references for this lesson.